

Session 2: AI for Coding – Practical Workflows and Demos

Elira Kuka and Tanner Regan

The George Washington University

Plan for Today: hands-on workflows

1. Coding with Claude Code overview

- ▶ Writing and debugging code from plain English *EK*
- ▶ “Vibe coding”: building a complete tool without writing a line of code *EK*

2. Data analysis demos with Stata

- ▶ Stata example with Claude Code from terminal *EK*
- ▶ Stata example with Claude Code from VS Code *TR*

3. Python in VS Code overview

- ▶ libraries and environments *TR*
- ▶ Jupyter extension *TR*

4. Data analysis demo with Python

- ▶ Python example with Claude Code from VS Code *TR*

Coding with Claude Code overview

Writing Code from Plain English

- ▶ You describe the task; AI writes the code — in any language (Stata, R, Python, Julia)
- ▶ This works even if you do not know the language
- ▶ Key skill: writing a good prompt
 - Describe inputs and outputs concretely: “I have a panel dataset with vars X, Y, Z...”
 - Specify what you want verified: “run it and show me the first 5 rows of output”
 - Be explicit about conventions: “use ‘i.year’ for year fixed effects in Stata”
- ▶ ▷ *Placeholder: side-by-side example of prompt → code*

Debugging Code with AI

- ▶ Paste the error message and ask: “what is wrong and how do I fix it?”
- ▶ With Claude Code in agentic mode: it sees the error itself and iterates automatically
- ▶ Particularly powerful for:
 - Obscure Stata/R errors with cryptic messages
 - Merge problems (wrong key, many-to-many, unmatched observations)
 - Output that looks right but is subtly wrong (off-by-one, wrong unit of observation)
- ▶ Important caveat: AI can fix the error while introducing a different bug
 - Always verify output against known benchmarks or summary statistics

“Vibe Coding”: Building Tools Without Writing Code

- ▶ Concept: describe what you want at a high level; AI writes, runs, and refines all the code
- ▶ You act as the product manager, not the programmer
- ▶ Examples relevant to economists:
 - “Build a script that downloads monthly CPI from FRED, merges with my wage data, and plots real wages by year”
 - “Create a do-file that runs my DiD specification with all the standard robustness checks and exports tables to LaTeX”
 - “Write a web scraper that pulls contract award data from USASpending.gov”
- ▶ When it works well; when to be cautious

Automating Repetitive Data Tasks

▶ Common tasks AI handles easily:

- Cleaning and reshaping messy data (wide ↔ long, renaming, recoding)
- Labeling variables and values from a codebook
- Merging multiple datasets on complex keys
- Generating a full set of summary statistics or balance tables

▶ Practical workflow for cleaning:

1. Show Claude the raw data structure (variable list, codebook, first rows)
2. Describe the cleaning steps in plain English
3. Review the output, ask for changes, iterate

▶ ▷ *Placeholder: example or screenshot*

Recap: Three Modes of Operation

▶ **Ask before edits** (safest):

- Claude proposes each file edit before making it; you approve or reject
- Best for: getting started, when you want full control, unfamiliar codebases

▶ **Plan mode** (recommended for most tasks):

- Claude writes a full plan in Markdown; you review and approve before anything runs
- Produces clean, commented code with an explicit audit trail
- Can view and revisit session history

▶ **Auto mode** (most powerful, most risk):

- Claude acts autonomously until the task is done
- Use for well-defined tasks; always check the output

Demo: Writing a Data Cleaning Script (Ask Mode)

- ▶ ▷ *Demo slide*
- ▶ Task: take a raw dataset, produce a clean analysis file
- ▶ Steps we will walk through:
 1. Point Claude Code at the project folder
 2. Write a minimum working example in “Ask before edits” mode
 3. Claude proposes each change; we approve
 4. Verify output matches expectations
- ▶ What to watch for:
 - Does the code run without errors?
 - Do summary statistics match what we expect?
 - Any implicit assumptions we need to check?

Demo: Cleaning Script Continued (Plan Mode)

- ▶ ▷ *Demo slide*
- ▶ Now run the same task in “Plan mode”:
 1. Ask Claude to write a plan before touching any files
 2. Review the plan in Markdown; request changes to the approach
 3. Approve; Claude executes and produces a clean, commented script
 4. Compare output to Ask-mode version: same result, better code
- ▶ Reviewing session history: how to audit what Claude did
- ▶ Ask Claude to update README with what was done (good practice)

When to Trust AI Code – and How to Verify It

- ▶ AI code can look completely correct and be wrong in subtle ways
- ▶ Verification checklist:
 - Does the code run without errors? (necessary, not sufficient)
 - Do observation counts make sense at each merge/collapse step?
 - Do means and distributions match codebook or prior literature?
 - Can you reproduce a known result (e.g., a published Table 1)?
 - Ask Claude: “what assumptions did you make?” – it will often reveal them
- ▶ General rule: trust AI code for *structure*; verify AI code for *content*

Building a Replication Package with Claude Code

- ▶ Claude Code can read your entire project and produce a replication package:
 1. Point Claude at the project directory
 2. Ask it to trace the data pipeline from raw files to final tables
 3. Ask it to write a master script that runs everything in order
 4. Ask it to write a README documenting data sources, software, and steps
- ▶ This task would normally take days — Claude does it in minutes
- ▶ You still need to verify: does running the master script reproduce the tables?
- ▶ AEA replication standards: what the package needs to include

Data analysis demos with Stata

Using Stata with Claude Code

- ▶ ▸ *Demo slide – Elira*
- ▶ Claude Code works with Stata via the terminal or VS Code (Stata Enhanced extension)
- ▶ What you can ask Claude to do:
 - Write a do-file from a plain-English description of the analysis
 - Debug a Stata error (paste the error; Claude explains and fixes it)
 - Translate a complex cleaning routine into clean, commented Stata code
 - Export regression tables to LaTeX (esttab, outreg2)
- ▶ Example: [PLACEHOLDER – data analysis example to demonstrate]

Stata in VS Code demo

- ▶ Very quickly - how to run stata from VS Code
 - ▶ Use the Stata Workbench Extension
 - ▶ Make sure the extension can 'find' stata
 - ▶ Work (mostly) as normal
 - ▶ Some things different, e.g. 'browse' needs to be done point and click.
 - ▶ Otherwise, Claude Code can work with Stata similarly to python
- ▷ *Stata & VS Code Demo slide*

Python in VS Code overview

Why Python?

► Claes Backman

Python for Economists: Common Tasks

- Data analysis with `pandas`: merging, collapsing, reshaping – analogous to Stata
- Econometrics: `statsmodels`, `linearmodels` (panel data, IV)
- Data collection: `requests`, `BeautifulSoup` for scraping (Session 3)
- Figures: `matplotlib`, `seaborn` – publication-quality plots
- Claude Code can translate your Stata code to Python (or vice versa) – useful for:
 - Running analyses you cannot do in Stata
 - Working with large datasets where Python is faster

Python basics

- ▶ Libraries and Environments
 - ▶ Anaconda manages Python installations and packages
 - ▶ Each project gets its own **environment**: a fixed set of packages and versions
 - Prevents conflicts between projects
 - Ensures others can reproduce your results exactly
 - ▶ Ask Claude Code to create and activate an environment for you
- ▶ Three ways to run Python code
 - **Script** (.py): run the full pipeline — best for production code
 - **Interactive window**: run .py scripts cell-by-cell — best of both worlds
 - **Jupyter notebook** (.ipynb): interactive cells, inline output — good for exploration

Python Setup in VS Code

- ▶ Required extensions: Python, Jupyter
- ▶ Managing environments:
 - Each project gets its own environment (set of packages + versions)
 - Prevents conflicts between projects
 - Ask Claude Code to create and activate an environment for you
- ▶ Running code:
 - **Jupyter notebook** (`.ipynb`): interactive cells, inline output — good for exploration
 - **Jupyter interactive window**: run `.py` scripts cell-by-cell — best of both worlds
 - **Script** (`.py`): run the full pipeline — best for production code

Data Analysis Demo with Python

I AM IMPRESSED

I AM IMPRESSED

- ▶ This example took about 2.5 days of my work
- ▶ Before Claude Code:
 - ▶ 2-4 weeks of my time
 - ▶ 1-2 months of very good PhD RA time
 - ▶ 2+ months of decent MS RA time
- ▶ Claude solution still more efficient and elegant
- ▶ More discussion later...

Python Exercise

Produced data (sampling frame for survey) and a [set of slides](#) in four steps:

1. Download data from Kigali City
2. Download data from Alpha Earth
3. Build a sample frame with visualization
4. Test an event study design with google satellite embeddings

Download data from Kigali City

1. Wrote [Prompt data scraping for Kigali.md](#) pasted into “plan mode”
 - ▶ I had to give permission on a few steps, e.g. searching URLs.
2. Claude generated the plan [kigali-city-data-mutable-frost.md](#) (~10 mins)
3. Executed the plan which created the script [download_kigali_gis_v0.py](#)
4. Ran the code without edits (using Jupyter window). Two issues:
 - ▶ Claude added packages to the yml but didn't update the conda environment. Chatted with Claude to identify and fix this.
 - ▶ I had daycare pickup and the code was still running. Claude adjusted the code so it would restart where I left off, rather than restarting completely.
5. Ran the updated script [download_kigali_gis.py](#) at home → success!
 - ▶ VS Code diff: right-click script.py “Select for Compare”, right-click script2.py “Compare with Selected”. Gives a clean side-by-side diff in the editor.

Download data from Google Earth Engine

1. Wrote [Prompt-data-scraping-Alpha-Earth.md](#) pasted into “plan mode”
2. Claude generated the plan [alpha-earth-data-witty-planet.md](#) (~10 mins)
3. Executed the plan to create [download_satellite_embedding_v0.py](#)
4. Ran the code without edits (using Jupyter window). issues:
 - ▶ Needed to add my GEE project code (ee-tannerregan-gwu)
 - ▶ More substantial errors appeared, I iterated with Claude to make [*_v1.py](#)
5. Ran the updated script successfully
6. Asked Claude to revise the code now that we had iterated
 - ▶ Simplify code lines from the debugging/iterating process and make sure well commented [download_satellite_embedding.py](#)

Download data from Google Earth Engine

1. Wrote [Prompt-data-scraping-Alpha-Earth.md](#) pasted into “plan mode”
2. Claude generated the plan [alpha-earth-data-witty-planet.md](#) (~10 mins)
3. Executed the plan to create [download_satellite_embedding_v0.py](#)
4. Ran the code without edits (using Jupyter window). issues:
 - ▶ Needed to add my GEE project code (ee-tannerregan-gwu)
 - ▶ More substantial errors appeared, I iterated with Claude to make [*_v1.py](#)
5. Ran the updated script successfully
6. Asked Claude to revise the code now that we had iterated
 - ▶ Simplify code lines from the debugging/iterating process and make sure well commented [download_satellite_embedding.py](#)

Check out VS Code Diff again!

Build a sample frame with visualization

- ▶ Wrote out three prompts (took me longer to get started with this exercise)
 - ▶ `Prompt-prepare-sample-frame.md` (writes draft)
 - ▶ wrote plan: `make-a-plan-based-zippy-sprout.md`
 - ▶ `Prompt-round2-prepare-sample-frame.md` (adjustments)
 - ▶ lost track of this plan (sorry)
 - ▶ `Prompt-round3-prepare-sample-frame.md` (added one new step)
 - ▶ wrote plan: `create-a-plan-that-witty-cosmos.md`
- ▶ The final python script is `prepare_sample_frame.py`

Test an event study with satellite embeds

- ▶ Wrote a prompt to run a couple example designs
 - ▶ `Prompt-event-study.md` was my initial prompt
 - ▶ This time I did many iterations and didn't keep track (sorry)
- ▶ The final python script was `event_study.py`
 - ▶ Claude also make the `4_slides_event_study.tex` that built the slides at the top of this section

Implications for RAs and research students

- ▶ My willingness to pay for PhD and MS student RAs is down
- ▶ Conditional on working with an RA, task allocation will change:
 - ▶ ↓ More menial tasks like undergrad RA (data collection, subjective classification, etc.)
 - ▶ ↑ More high level conceptual tasks (design survey questionnaires, design data cleaning / empirical strategy)
- ▶ This has implications for students
 - ▶ They still need to know the fundamentals of coding to make CC productive, but training opportunities will become more scarce
 - ▶ Otherwise, RAships that focus more on high level conceptual tasks will probably help make them better researchers. But IMO requires higher RA ability and trust

Wrapping Up

Key Takeaways from Sessions 1 & 2

1. AI handles the coding mechanics — your job is to verify the output and drive the analysis
2. Use Plan mode for any non-trivial task; it gives you an audit trail and catches mistakes early
3. Git + Claude Code = version-controlled, reproducible research without learning Git commands
4. CLAUDE.md files carry your preferences across sessions so you do not repeat yourself
5. Replication packages are now a hours-long task, not a weeks-long one
6. The critical skill is *verification*: check outputs, not just that the code runs

Things to Try Before the Next Session

- ▶ Set up VS Code with Claude Code, Git, and Python (use the setup guide)
- ▶ Ask Claude to write a small data task in your preferred language — verify the output
- ▶ Initialize a Git repo for one of your current projects
- ▶ Write a CLAUDE.md file for that project
- ▶ Coming up in Session 3: Finding and Collecting Data
 - Identifying datasets, scraping, APIs, parsing PDFs at scale

Questions?

- ▶ Questions on anything from Sessions 1 or 2?
- ▶ What tasks in your own research would you most like to try this on?